Order Book Performance Analysis

1. Which implementation is the fastest?

- add_order(), cancel_order() and get_bbo(): Implementation C is fastest
    1. Finding a single subtraction and jumps there in 1 cpu cycle.
    2. Array is a contiguous block of memory leading to less cache misses.
    3. bestBidPrice and bestAskPrice is already a variable so looking it in array of bids and ask takes O(1) time complexity or 1 cpu cycle

2. Branch Prediction : Implementation B has most branch misses

- Every time an order is push or pop from the queue the inner implementation of priority queue has to make decision if it should move the element up or down the tree
- The get_bids() and get_asks() runs 10000 times in the benchmarks which means we are deep copying the priority queue 10000 times and popping from them 100000 times ultimately making more decision making

3. Cache Behaviour : Implementation B has most cache misses

- There is exponential jumping across the memory in the priority queue. For example if i am looking at 5000th element the next element will be in like 10000th element making the cache stale
- Also, to make this even worse we are deep copying the full priority queue during the get_bids() and get_asks().

4. Page fault: Implementation A and C has most page faults

- In implementation A and C we are pre-allocating a massive amount of orderpool at startup.

5. Production Choice : Implementation C

- Latency: sub-15 nanoseconds execution across all operations.
- Complexity: does not rely on rebalancing algorithms dual state tracking with lazy deletions and auxiliary hash tables.

- CPU cache behaviour: arrays are continuous in memory leading to less cache misses compared to other implementations.
- Predictability: It has strictly zero runtime allocations on the trading path. Every operation is constant-time O(1) memory manipulation. Its 99th-percentile latency is almost identical to its median.
- Memory: It allocates all 10,001 price slots upfront at startup (~320 KB for the array + the order pool).
- Maintainability: There is simple fix size array and basic pointer linking and unlinking no complex tree rotation.

6. What would we try next? : Cache-Line Alignment (alignas(64)) & Structure Packing

- Currently, PriceLevel and OrderNode are of arbitrary sizes which can split across multiple cache line boundaries. Padding and packing PriceLevel can help me fit exactly two PriceLevel in 1 cache line each being 32 bytes and ensuring 0 cache line penalty.

7. Results Table

Metric	Impl A (std::map)	Impl B (priority_queue)	Impl C (Direct Index Array)
Total Runtime (ms)	110.96 ms	8,297.26 ms	21.34 ms
ns / add_order	81.50 ns	41.00 ns	12.51 ns
ns / cancel_order	68.68 ns	12.79 ns	7.22 ns
ns / get_bbo	0.267 ns	3.965 ns	0.265 ns
IPC	0.48	0.25	0.59
L1-dcache Misses	57.90 Million	3.80 Billion	15.09 Million
Branch Misses	38.64 Million	252.10 Million	5.67 Million
Page Faults	971,250	696,661	971,208